



FORMATION PROMETHEUS & GRAFANA

Guide stagiaire — Jour 1

Voir : architecture et collecte

Sommaire

Voir : architecture et collecte	2
Le programme du jour	2
Le fil rouge	2
Les services du lab	2
Rappels	3
Exercice 1.1 — Démarrer et vérifier	3
Exercice 1.2 — Lire une page /metrics	4
Exercice 1.3 — L'interface de Prometheus	4
Exercice 1.4 — Premières requêtes	4
Rappels — configuration	5
Exercice 1.5 — Changer le rythme	5
Exercice 1.6 — Casser pour comprendre	5
Exercice 1.7 — Découverte par fichier	5
Exercice 1.8 — Relabeling (bonus)	6
Rappels — exporters	6
TP 1 — Node Exporter : de la machine à Prometheus	6
Rappels — instrumentation	8
TP 2 — Instrumentation et services tiers	8
Avant de partir	10
Quiz de fin de journée	10

Voir : architecture et collecte

Ce guide contient les énoncés des exercices du jour, quelques rappels et de la place pour vos réponses. Les corrigés sont donnés en séance, après un temps de recherche : jouez le jeu.

Le programme du jour

Heure	Séquence
9h00	Accueil, tour de table
9h30	Pourquoi l'observabilité
10h05	Architecture de Prometheus, place de Grafana
11h00	Installation et prise en main — exercices 1.1 à 1.4
11h50	Configuration de Prometheus — exercices 1.5 à 1.8
14h00	Les exporters
14h30	TP 1 — Node Exporter
15h45	Instrumenter son application
16h15	TP 2 — Instrumentation et services tiers
17h15	Récap et quiz

Le fil rouge

Une boutique en ligne, `shop-api`, en deux instances, instrumentée avec `prometheus_client`, avec un générateur de trafic qui simule des clients. Tout au long des trois jours, on la surveille, on la casse, on se fait prévenir.

Les services du lab

Service	URL locale	Rôle
Grafana	<code>http://localhost:3000</code>	Visualisation — <code>admin</code> / <code>formation</code>
Prometheus	<code>http://localhost:9090</code>	Collecte, stockage, PromQL
Alertmanager	<code>http://localhost:9093</code>	Routage des alertes (jour 3)
Inbox	<code>http://localhost:8080</code>	Boîte de réception des notifications
<code>shop-api-1 / -2</code>	<code>http://localhost:5001 · :5002</code>	La boutique — <code>/metrics</code>
Node Exporter	<code>http://localhost:9100/metrics</code>	Métriques machine
Blackbox Exporter	<code>http://localhost:9115</code>	Sondes

Service	URL locale	Rôle
Pushgateway	http://localhost:9091	Batches
Redis Exporter	http://localhost:9121/metrics	Service tiers

Sur Codespaces, utilisez l'onglet **Ports** de VS Code pour ouvrir chaque service.

Commandes : `./lab.sh up | status | check | reload | logs <service>` (Windows : `.\lab.ps1`).
Après **chaque** modification de `prometheus/prometheus.yml` : `./lab.sh check` puis `./lab.sh reload` .

Rappels

Pull. Prometheus va chercher (scraper) les métriques sur chaque cible en HTTP, à intervalle fixe (15 s par défaut). Une cible qui ne répond pas, c'est `up == 0` .

Le modèle de données. Métrique + labels = série. `http_requests_total{method="GET", route="/api/products", status="200"}` . Chaque combinaison de valeurs de labels est une série distincte.

Les quatre types.

Type	Comportement	Exemple	Usage
Counter	ne fait que monter	<code>http_requests_total</code>	<code>rate()</code> , jamais brut
Gauge	monte et descend	<code>shop_cart_items</code>	valeur brute
Histogram	buckets cumulatifs <code>le=</code>	<code>http_request_duration_seconds</code>	<code>histogram_quantile()</code>
Summary	quantiles côté client	<code>go_gc_duration_seconds</code>	lecture directe

Le format d'exposition.

```
# HELP http_requests_total Nombre total de requêtes HTTP reçues
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/api/products",status="200"} 51.0
```

Exercice 1.1 — Démarrer et vérifier

1. Lancez la stack : `./lab.sh up` (ou `.\lab.ps1 up` , ou créez votre Codespace).
2. Vérifiez que tous les conteneurs sont `Up` : `./lab.sh status` .
3. Ouvrez Prometheus, Grafana, Alertmanager et l'Inbox dans quatre onglets.
4. Dans Prometheus, **Status** → **Target health** : combien de cibles sont `UP` ? Lesquelles ?

Réponse :

Exercice 1.2 — Lire une page /metrics

Ouvrez `http://localhost:5001/metrics`.

1. Trouvez une métrique de chaque type (counter, gauge, histogram, summary). Notez leur nom.
2. Pour `http_request_duration_seconds`, combien de buckets ? Quelle est la borne du dernier ?
3. Quelle est la version de l'application ? Où est-elle stockée ?
4. Que vaut `shop_revenue_euros_total` ? Rechargez la page : elle a bougé ?

Réponses :

Astuce : `Ctrl+F` sur `# TYPE`.

Exercice 1.3 — L'interface de Prometheus

1. **Status → Configuration** : retrouvez le `scrape_interval` global.
2. **Status → Runtime & build information** : quelle version ? Depuis quand tourne-t-il ?
3. **Status → TSDB status** : combien de séries en mémoire ? Quelle métrique a le plus de séries ?
4. Onglet **Query** : tapez `up`, exécutez, puis passez en onglet **Graph**.

Réponses :

Exercice 1.4 — Premières requêtes

Dans l'onglet Query :

1. `http_requests_total` : combien de séries ?
2. Ne gardez que les requêtes de `shop-api-2` (label `instance`).
3. Ne gardez que les erreurs (codes 4xx et 5xx) : label `status`, expression régulière.
4. Combien de requêtes `POST` sur `/api/checkout` depuis le démarrage de `shop-api-1` ?

Vos requêtes :

Astuce : `{label="valeur"}`, `!=`, `=~"regex"`. Les regex sont ancrées : `5..` *matche exactement trois caractères*.

Rappels — configuration

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
rule_files:
  - rules/*.yaml
alerting:
  alertmanagers:
    - static_configs:
      - targets: ["alertmanager:9093"]
scrape_configs:
  - job_name: shop-api
    static_configs:
      - targets: ["shop-api-1:5000", "shop-api-2:5000"]
      labels:
        env: formation
```

Valider : `./lab.sh check` . Recharger : `./lab.sh reload` (OU `curl -X POST localhost:9090/-/reload`).
Les chemins de fichiers sont relatifs au dossier `prometheus/` (monté dans `/etc/prometheus`).

Exercice 1.5 — Changer le rythme

1. Pour le job `shop-api` uniquement, passez le `scrape_interval` à 5 s.
2. Validez, rechargez.
3. Vérifiez dans **Status** → **Target health** (colonne *Last scrape*) et avec `prometheus_target_interval_length_seconds{quantile="0.99"}` .
4. Remettez 15 s (important pour demain).

Exercice 1.6 — Casser pour comprendre

1. Introduisez une erreur dans `prometheus.yml` (un `:` en trop, un tiret manquant).
2. Rechargez **sans** valider. Que se passe-t-il ? Regardez `./lab.sh logs prometheus` et la métrique `prometheus_config_last_reload_successful` .
3. Réparez, rechargez, vérifiez que la métrique repasse à 1.

Ce que vous avez observé :

Exercice 1.7 — Découverte par fichier

La boîte de réception (`inbox`) expose une page `/metrics` sur le port 8080. Ajoutez-la à Prometheus **sans** la lister dans `prometheus.yml` :

1. Ajoutez un job `file-sd` avec `file_sd_configs` qui lit `targets/*.yaml` (et `*.json`),
`refresh_interval: 30s` .
2. Créez `prometheus/targets/extra.yaml` avec la cible `inbox:8080` et un label `tier: outils` (voir l'exemple dans `prometheus/targets/README.md`).

3. Rechargez une fois (pour le nouveau job). Vérifiez ensuite que la cible apparaît : les fichiers de cibles sont relus automatiquement.

Exercice 1.8 — Relabeling (bonus)

L'exporter Redis (`redis-exporter:9121`) expose ses propres métriques Go (`go_*` , `process_*` , `promhttp_*`) en plus des métriques Redis. Ajoutez un job `redis-light` qui scrape cet exporter en jetant ces métriques internes avec `metric_relabel_configs` (`action: drop`). Comparez le nombre de séries avec le job `redis` du TP 2 : `count by (job) ({job=~"redis.*"})` .

Astuce : `source_labels: [__name__]` et une `regex` avec des `|` .

Rappels — exporters

Un exporter est un adaptateur : il interroge un système (noyau Linux, base de données, équipement réseau) et expose une page `/metrics` . Node Exporter (9100) pour Linux, Blackbox (9115) pour les sondes externes, Pushgateway (9091) pour les batchs trop courts pour être scrapés.

Dans un conteneur, le Node Exporter ne voit la machine hôte que si on lui monte `/proc` , `/sys` et `/` (c'est fait dans `docker-compose.yml`). Sur Docker Desktop, « l'hôte » est la VM Linux.

TP 1 — Node Exporter : de la machine à Prometheus

Situation. Vous venez de recevoir un serveur. Avant d'y déployer quoi que ce soit, vous voulez ses signes vitaux dans Prometheus, et pouvoir y ajouter vos propres indicateurs avec un simple script.

Partie 1 — Brancher l'exporter

1. Dans `prometheus/prometheus.yml` , ajoutez un job `node` qui scrape `node-exporter:9100` .
2. Validez, rechargez, vérifiez la cible dans Target health.
3. Requête : `node_uname_info` . Quel noyau ? Quel nom de machine ?

Réponse :

Partie 2 — Les indicateurs de base

Écrivez une requête pour chaque indicateur (les métriques et fonctions nécessaires sont données) :

1. Uptime en secondes : `time()` et `node_boot_time_seconds` .
2. Mémoire disponible en % : `node_memory_MemAvailable_bytes` , `node_memory_MemTotal_bytes` .
3. Espace disque utilisé en % sur `/` : `node_filesystem_avail_bytes` , `node_filesystem_size_bytes` , label `mountpoint` .
4. Charge moyenne 1 min : `node_load1` . Comparez au nombre de cœurs : `count(node_cpu_seconds_total{mode="idle"})` .

5. CPU utilisé en % (formule donnée, à recopier et à **comprendre**) : `100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))` .

Lancez `./lab.sh chaos cpu 120` et observez la courbe CPU dans l'onglet Graph.

Vos requêtes :

Astuce : une division entre deux métriques marche si elles ont exactement les mêmes labels.

Partie 3 — Configurer les collectors

1. Le collector `processes` est désactivé par défaut. Activez-le : ajoutez `--collector.processes` aux `command` du service `node-exporter` dans `docker-compose.yml`, puis `docker compose up -d node-exporter` .
2. Vérifiez l'apparition de `node_processes_state` et `node_processes_threads` .
3. Désactivez un collector inutile pour vous (par exemple `--no-collector.arp`) et vérifiez que `node_arp_entries` disparaît de `/metrics` .

Astuce : `docker run --rm prom/node-exporter:v1.12.1 --help` liste tous les collectors.

Partie 4 — Le textfile collector

Vous avez un script de sauvegarde en cron. Vous voulez savoir quand il a tourné pour la dernière fois.

1. Créez `node-exporter/textfile/backup.prom` avec une gauge `backup_last_run_timestamp_seconds` (valeur : le timestamp actuel, `date +%s`) et une gauge `backup_files_total` . Le format est celui d'une page `/metrics` (lignes `# HELP` , `# TYPE` , puis la valeur), avec un saut de ligne final.
2. Vérifiez sur `http://localhost:9100/metrics` puis dans Prometheus.
3. Calculez « il y a combien de temps » : `time() - backup_last_run_timestamp_seconds` .

Astuce Windows : `"backup_last_run_timestamp_seconds`

`$([DateTimeOffset]::UtcNow.ToUnixTimeSeconds())" | Set-Content node-exporter/textfile/backup.prom` . Si rien n'apparaît : regardez `node_textfile_scrape_error` .

Partie 5 — Vue d'ensemble

1. Quel est le disque (`mountpoint`) le plus rempli ? `topk(1, ...)` sur la requête de la partie 2.
2. Combien d'octets par seconde entrent sur les interfaces réseau (hors `lo`) ?
`rate(node_network_receive_bytes_total{device!="lo"}[5m])` .
3. Dans Grafana, **Explore** : collez la requête CPU, regardez les 30 dernières minutes, repérez le chaos CPU.
4. Bonus : importez le dashboard communautaire *Node Exporter Full* (Dashboards → New → Import, ID `1860`). Nécessite un accès à `grafana.com`.

Notes :

Rappels — instrumentation

```
from prometheus_client import Counter, Gauge, Histogram

ORDERS = Counter("shop_orders_total", "Commandes validées", ["payment_method"])
STOCK = Gauge("shop_stock_units", "Unités en stock par produit", ["product"])
DURATION = Histogram("http_request_duration_seconds", "Durée", ["route"])

ORDERS.labels(payment_method="card").inc()
STOCK.labels(product="clavier").set(84)
DURATION.labels(route="/api/checkout").observe(0.083)
```

Nommage : `snake_case` , préfixe métier, unité de base en suffixe (`_seconds` , `_bytes`), `_total` pour les counters. Labels : jamais de valeur non bornée (identifiant, IP, email, URL brute, terme de recherche). **RED** pour un service : Rate, Errors, Duration.

TP 2 — Instrumentation et services tiers

Situation. Le directeur commercial veut savoir quelles fiches produit sont les plus consultées. L'équipe infra veut surveiller Redis. Le support veut être prévenu si le site est inaccessible depuis l'extérieur. Et il y a ce batch de sauvegarde nocturne...

Partie 1 — Une métrique métier dans le code

1. Dans `apps/shop-api/app.py` , déclarez un Counter `shop_product_views_total` avec un label `product` (cherchez le `TOD0`).
2. Incrémentez-le dans la route `/api/products/<product>` , au bon endroit.
3. Reconstituez et redémarrez : `docker compose up -d --build shop-api-1 shop-api-2` .
4. Vérifiez sur `/metrics` , puis dans Prometheus : quel produit est le plus consulté ? `topk(1, sum by (product) (shop_product_views_total))` .

Réponse :

Question : pourquoi placer l'incrément après la vérification `if product not in PRODUCTS` ?

Partie 2 — Un exporter tiers : Redis

1. Ajoutez le job `redis` (cible `redis-exporter:9121`). Validez, rechargez.
2. Requêtes : `redis_up` , `redis_connected_clients` , `rate(redis_commands_processed_total[5m])` .
3. Commande Redis la plus utilisée : `topk(3, rate(redis_commands_total[5m]))` .

Réponse :

Partie 3 — Sondes externes avec Blackbox

Vous voulez vérifier depuis l'extérieur que `http://shop-api-1:5000/health`, `http://shop-api-2:5000/health`, `http://grafana:3000/api/health` répondent 2xx, et que `https://prometheus.io` est joignable (accès Internet nécessaire ; sinon, ignorez cette cible).

1. Ajoutez un job `blackbox-http`. Squelette à compléter (trois règles de relabeling) :

```
- job_name: blackbox-http
  metrics_path: /probe
  params:
    module: [http_2xx]
  static_configs:
    - targets:
      - http://shop-api-1:5000/health
      # ...
  relabel_configs:
    # 1. copier __address__ dans __param_target
    # 2. copier __param_target dans instance
    # 3. remplacer __address__ par blackbox-exporter:9115
```

2. Requêtes : `probe_success`, `probe_duration_seconds`, `probe_http_status_code`.
3. Arrêtez `shop-api-2` (`docker compose stop shop-api-2`) et observez `probe_success` et `up`. Que remarquez-vous sur `up` ? Redémarrez (`docker compose start shop-api-2`).
4. Bonus : `probe_ssl_earliest_cert_expiry - time()` pour `prometheus.io` : dans combien de jours le certificat expire-t-il ?

Votre relabeling :

Observation sur `up` :

Astuce : `source_labels`, `target_label`, `replacement`. Le paramètre d'URL `target` s'écrit `__param_target` dans les labels.

Partie 4 — Un batch et la Pushgateway

1. Lancez le batch : `./lab.sh batch`. Regardez `http://localhost:9091`.
2. Ajoutez le job `pushgateway` (cible `pushgateway:9091`) avec `honor_labels: true`. Validez, rechargez.
3. Requêtes : `backup_duration_seconds`, `backup_size_megabytes`, et l'ancienneté de la dernière sauvegarde : `time() - backup_last_success_timestamp_seconds`.
4. Sans `honor_labels`, que se passerait-il ? Essayez, comparez les labels `job` et `instance`.

Observation :

Partie 5 — Chasse à la cardinalité (bonus)

1. Les dix métriques qui ont le plus de séries : `topk(10, count by (__name__) ({__name__=~"."+"}))`.
2. Séries au total : `prometheus_tsdb_head_series`. Séries du job `redis` : `count({job="redis"})`.

3. Si la boutique passait à 200 routes et 50 instances, combien de séries pour `http_request_duration_seconds_bucket` ? Est-ce raisonnable ? Que feriez-vous ?

Réponse :

Avant de partir

- `prometheus.yml` doit contenir les jobs : `prometheus` , `shop-api` , `node` , `file-sd` , `redis` , `pushgateway` , `blackbox-http` . Tous UP dans Target health.
- Ne faites pas `./lab.sh reset` : on veut de l'historique pour demain. Sur Codespaces, le Codespace peut s'arrêter tout seul, les données restent.

Quiz de fin de journée

1. Pull ou push : Prometheus fait quoi, et pourquoi ?
2. Que vaut un Counter brut ?
3. Trois labels qu'on ne met jamais sur une métrique.
4. Que fait `honor_labels: true` ?
5. Comment vérifier une configuration avant de recharger ?
6. Pourquoi `up` reste à 1 quand une cible du blackbox tombe ?
7. Le Node Exporter dans Docker Desktop mesure quoi ?
8. Quel est le suffixe d'un counter ? D'une durée ?